Rockchip RK2118 RT-Thread Quick Start

ID: RK-JC-YF-589

Release Version: V1.0.4

Release Date: 2024-08-16

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2024. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

The document presents the basic usage of Rockchip RK2118 RT-Thread SDK, aiming to help engineers get started with RK2118 RT-Thread SDK faster.

Product Version

Chipset	Kernel Version
RK2118	RT-Thread v4.1.x

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Date	Version	Author	Revision History
2024-02-26	V1.0.0	Roger Hu	Initial version
2024-03-15	V1.0.1	Roger Hu	Add operation debugging description
2024-08-16	V1.0.4	Roger Hu	Update compilation instructions

Contents

Rockchip RK2118 RT-Thread Quick Start

1. Set up the Development Environment
2. Project Directory
3. Project Building Configuration
   - 3.1 CPU Firmware Compilation
     - 3.1.1 Automated Compilation and Packaging
     - 3.1.2 Compilation Results Cleanup
     - 3.1.3 Manual Compilation and Packaging
       - 3.1.3.1 Module Configuration
       - 3.1.3.2 Compilation
       - 3.1.3.3 Generate root.img
       - 3.1.3.4 Packing
   - 3.2 DSP Firmware Compilation
   - 3.3 Firmware Format
     - 3.3.1 Firmware.img
     - 3.3.2 update.img
   - 3.4 File System Description
4. Firmware Flash
   - 4.1 Note
   - 4.2 Windows Upgrade Tool
     - 4.2.1 Firmware.img
     - 4.2.2 update.img
   - 4.3 Linux Upgrade Tool and Command
     - 4.3.1 UART Programming
     - 4.3.2 USB Programming
5. Run and Debug
   - 5.1 System Start
   - 5.2 System Debug
6. Development Guide

1. Set up the Development Environment

The recommended build environment for this SDK is a 64-bit version of Ubuntu 20.04 or Ubuntu 18.04. It has not been tested on other Linux distributions, so it is recommended to install the same distribution as RK developers.

The compilation tools used are SCons + GCC, as officially recommended by RT-Thread. SCons is an open-source build system written in Python, and the GCC cross-compiler is provided by ARM officially. You can directly use the following commands to install all the required tools:

```
sudo apt-get install gcc-arm-embedded scons clang-format astyle libncurses5-dev
build-essential python-configparser
```

Download the compiler from the ARM official website, and specify the path to the toolchain via environment variables, as detailed below:

```
wget https://developer.arm.com/-/media/Files/downloads/gnu/13.2.rel1/binrel/arm-
gnu-toolchain-13.2.rel1-x86_64-arm-none-eabi.tar.xz
tar xvf arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-eabi.tar.xz
export RTT_EXEC_PATH=/path/to/toolchain/arm-gnu-toolchain-13.2.Rel1-x86_64-arm-
none-eabi/bin
```

Or use the compiler included in the initial release package of the SDK: arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-eabi.tar.xz, as detailed below:

```
tar -xvf arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-eabi.tar.xz
export RTT_EXEC_PATH=/path/to/toolchain/arm-gnu-toolchain-13.2.Rel1-x86_64-arm-
none-eabi/bin
```

2. Project Directory

The following is the main SDK directory:

```
├── applications           # Rockchip application demo source
├── AUTHORS
├── bsp                    # Chip related source code
│   ├── rockchip
│   │   ├── common
│   │   │   ├── drivers    # Universal driver of Rockchip OS adaptation layer
│   │   │   ├── hal        # Rockchip HAL (Hardware abstraction layer)
implementation
│   │   │   └── tests      # Rockchip driver test code
│   │   ├── rk2118         # RK2118 main directory
│   │   │   ├── board      # Board level configuration
│   │   │   ├── build      # Build main directory and store the intermediate
files
│   │   │   ├── drivers    # RK2118 private driver directory
```

```
│   │   │   ├── Image       # Stores firmware
│   │   └── tools           # Rockchip commonly used tools
├── ChangeLog.md
├── components              # Various components of the system, including file
system, shell and framework layer and other drivers
│   ├── hifi4
|   |   ├── dsp             # DSP code
|   |   ├── rtt             # DSP related code running on mcu
|   |   ├── shared          # mcu/dsp common code
|   |   └── tools           # dsp firmware generation tool
├── documentation           # RT-Thread official documentation
├── examples                # RT-Thread example program and test code
├── include                 # RT-Thread official header file directory
├── Kconfig
├── libcpu
├── LICENSE
├── README.md
├── README_zh.md
├── RKDocs                 # Rockchip documents
├── src                    # RT-Thread kernel source code
├── third_party            # Directory of third-party code added by Rockchip
└── tools                  # RT-Thread official tool directory, including
menuconfig and building scripts
```

3. Project Building Configuration

3.1 CPU Firmware Compilation

RT-Thread utilizes SCons for compile control. SCons is an open-source build system written in Python, similar to GNU Make. Unlike traditional Makefile approaches, it uses SConstruct and SConscript files as substitutes. These files are also Python scripts, allowing the use of standard Python syntax for scripting. Therefore, in SConstruct and SConscript files, one can invoke the Python standard library to perform various complex operations, not limited to the rules set by Makefiles.

3.1.1 Automated Compilation and Packaging

To simplify the compilation process, we created a `build.sh` script to encapsulate the compilation and packaging commands.

Compile all CPU firmware:

```
cd bsp/rockchip/rk2118

# Compile for board: RK_AUTOMOTIVE_DSP_DEMO_RK2118M_V10_2024-01-25
./build.sh board/adsp_demo/setting.ini
# Generate: Image/Firmware.img

# Compile for board: RK_EVB1_RK2118G_V10_20240219
./build.sh board/evb/package-file
# Generate: Image/update.img

# Compile for board: RK_EVB1_RK2118G_V20_20240424
./build.sh board/evb1_v20/package-file
# Generate: Image/update.img
```

Then upgrade the generated Firmware.img or update.img

Compile the firmware for a single CPU:

```
# Take the EVB1 V20 board: RK_EVB1_RK2118G_V20_20240424 as an example

# CompileCPU0
./build.sh board/evb/package-file rtt0
# Generate: rtt0.bin, then
./mkimage.sh board/evb/package-file
# Generate: Image/rtt0.img

# You can view more detailed compilation instructions using the following
command
./build.sh

# Note that XIP is enabled by default in rtconfig.py.
# Therefore, when compiling non-XIP firmware (for SPI NAND/eMMC), you need to
manually disable XIP. Otherwise, the compiled firmware will run abnormally.
# Modify the bsp/rockchip/rk2118/rtconfig.py file, find 'XIP', and change it to
'N'.
```

Then, separately upgrade the generated rtt0.img.

Taking the `adsp_demo` board as an example, since we only provide the configuration for `cpu0` for this board, you can only select `cpu0`. The specifics are as follows:

```
./build.sh board/adsp_demo/setting.ini
```

This command actually performs the following operations:

1. Locate the default configuration for CPU0 of the specified board at the path: `board/adsp_demo/defconfig`, and use it to overwrite the current default `.config` file in the menuconfig directory.
2. Run the command `scons menuconfig`. A menuconfig configuration window will pop up, allowing you to modify settings as needed. If you wish to use the default configuration, you can simply exit.
3. Execute `scons -j$(nproc)` to compile the CPU0 firmware `rtt0.bin`. If configuration is present, it will also compile `cpu1` firmware `rtt1.bin` and the `recovery.bin` for OTA upgrades.
4. Run the `./mkroot.sh` command to generate the `root.img` image, provided that `setting.ini` has the necessary configuration.

5. Use `mkimage.sh` to package and generate the final firmware image as `Firmware.img` or `update.img`.

After running the above commands, the following files will be generated in the `Image` directory.

```
-rw-r--r-- 1 rk rk 1638400 Apr 11 02:41 Firmware.img           # For automotive SPI Nor
-rw-rw-r-- 1 rk rk 2750993 Apr 11 02:48 update.img             # For consumer SPI Nor/Nand/eMMC
-rw-r--r-- 1 rk rk 4325376 Mar 27 03:23 root.img              # Root filesystem image (optional)
```

3.1.2 Compilation Results Cleanup

The SCons build system by default uses MD5 to determine whether files need to be recompiled. If the content of your file has not changed, but only the timestamp has (for example, updated via the `touch` command), it will not recompile that file or its dependencies. Additionally, if you modify unrelated content, such as code comments, it will compile but not link, because the content of the obj file has not changed. Therefore, if you encounter issues where modifications do not take effect during the development process, it is recommended to perform a cleanup before compiling, using the following command:

```
scons -c
```

If anomalies still occur after performing the above cleanup, you can forcefully delete all intermediate files using the following command:

```
rm -rf build
```

For other SCons commands, you can refer to the help or documentation.

```
scons -h
```

3.1.3 Manual Compilation and Packaging

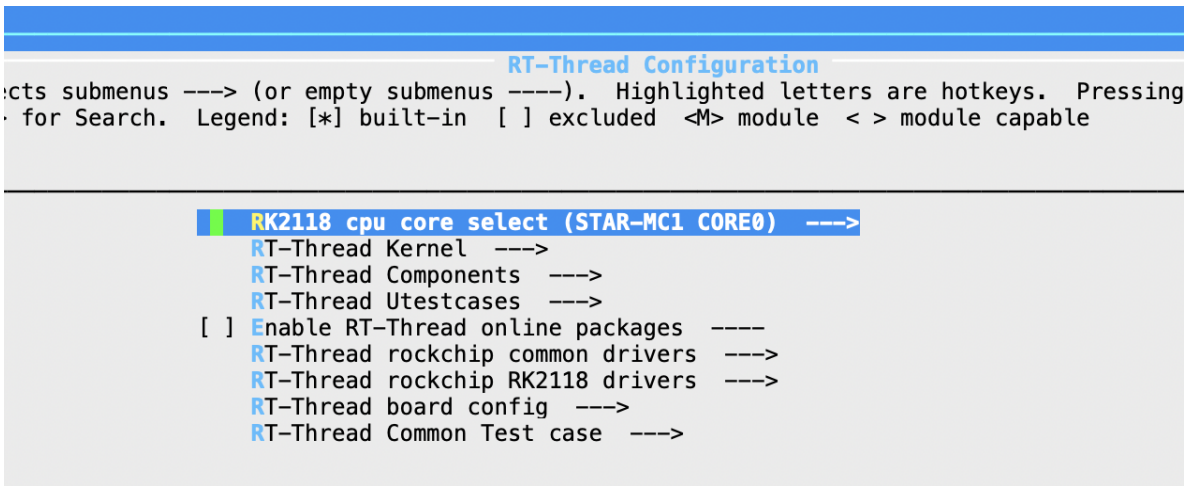
Although using the `build.sh` script for one-click compilation is convenient, it updates the `rtconfig.h` module configuration file each time, which causes the entire CPU project to be recompiled, making it slightly slower than manual compilation. Therefore, manual compilation is often used during development to speed up the process. The manual compilation steps can be divided into four phases: module configuration, compilation, root.img generate and packaging. If your module configuration does not need changes, you can skip the module configuration step. Similarly, if the CPU firmware does not need changes, you can skip the compilation step and proceed directly to packaging.

3.1.3.1 Module Configuration

The purpose of module configuration is to select the necessary modules based on the actual needs of the product. This can be done using the following command:

```
cd bsp/rockchip/rk2118
# First, use a base configuration from your board to overwrite the default
configuration file: .config
cp board/adsp_demo/defconfig .config
# Load the default configurations from .config and launch the graphical
configuration interface
scons --menuconfig
```

At this point, the following interface will pop up. You can enable or disable various modules according to the needs of the product. Exiting and saving the configuration will overwrite the `.config` file, and at the same time, automatically generate an `rtconfig.h` file. These two files contain all the selected configurations, and only the `rtconfig.h` file will be used in the final compilation.



In the above figure, the first item is to select the target CPU you want to compile for. RK2118 has two CPUs: cpu0 and cpu1. The next three items are the standard configurations of RT-Thread, followed by RT-Thread's network package, and the rest are the driver configurations and test cases for BSP.

Common operations of the menuconfig tool are as follows:

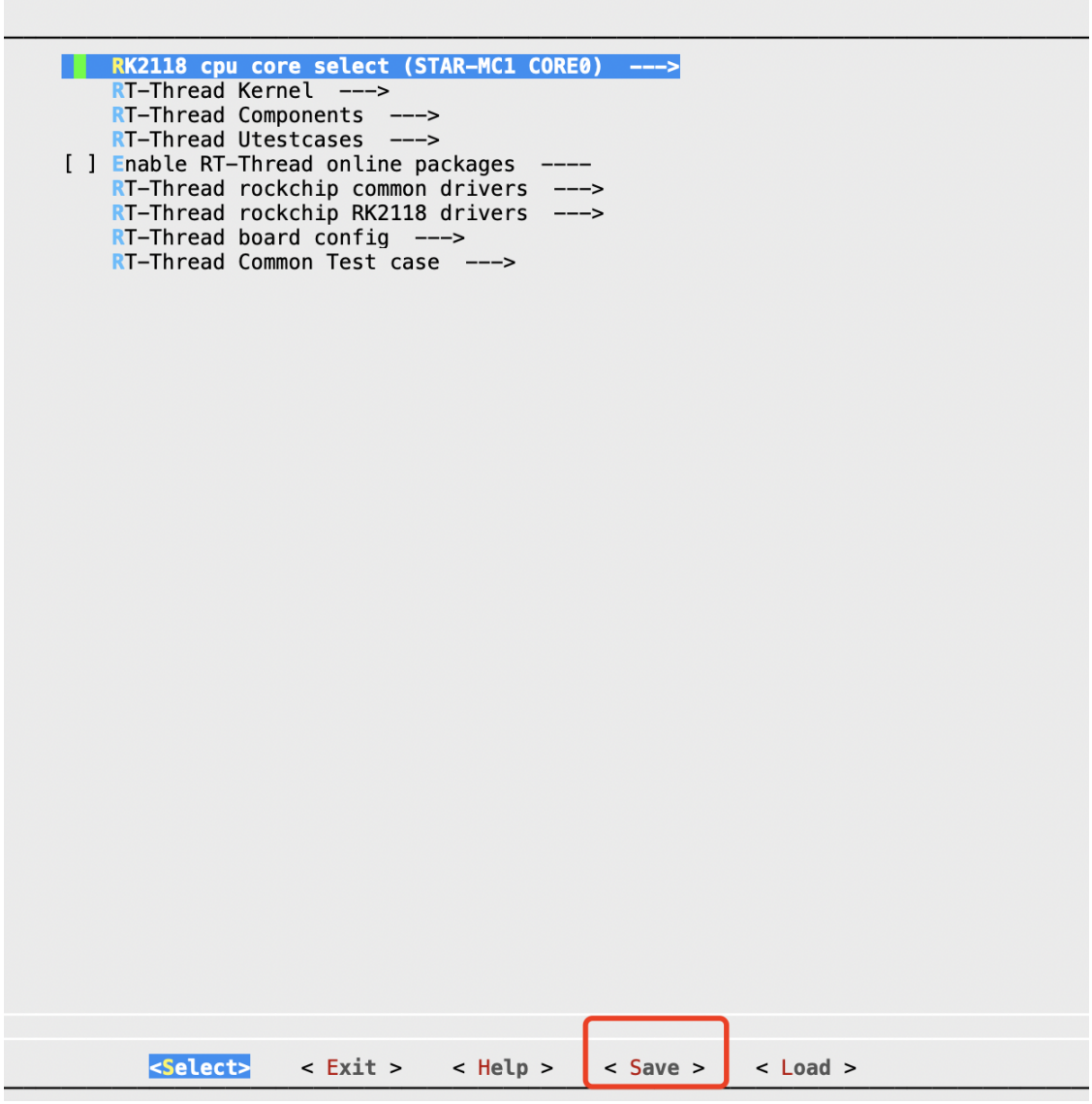
- Up/Down arrows: Move cursor
- Enter: Enter a submenu
- ESC key: Return to the previous menu or exit
- Space, Y key, or N key: Enable/disable [*] configuration options
- Question mark: Brings up a help menu for the highlighted option (to exit the help menu, press Enter)
- Slash (/) key: Search for configuration items

  Each board-level directory contains at least one default configuration file `defconfig`, which is the default configuration for cpu0. If the board level supports dual-cpu boot, there will also be a `cpu1_defconfig`, which is the default configuration for cpu1. When submitting default module configurations, we recommend submitting them to the board-level directory and maintaining this naming convention as much as possible to prevent the automatic compilation script from failing. Do not directly submit to `.config`, so that configurations across different board levels do not overwrite each other. The method is as follows:

```
  # Assume that you have previously run scons --menuconfig and saved the
configuration upon exiting
  cp .config board/xxx/defconfig       # xxx is the name of your board. If it is
the configuration for cpu1, please change it to cpu1_defconfig
```

Please note that as mentioned earlier, only `rtconfig.h` is involved in the compilation, not `.config`. Therefore, if you are not prompted to save the configuration when exiting the `menuconfig` graphical interface, it is because you have not modified any options during this `menuconfig` session. In this case, `rtconfig.h` will not be regenerated, and your `.config` might have been manually overwritten and no longer matches `rtconfig.h`, leading to your current `.config` not being effective during compilation. There are two ways to circumvent this issue, and you can choose one based on your need:

1. Do not exit the `menuconfig` graphical interface directly; instead, save the configuration first and then exit. You can manually save the configuration using this option:



2. After exiting, execute the command `scons --useconfig=.config`. This will forcefully regenerate `rtconfig.h`.

3.1.3.2 Compilation

Compiling the CPU firmware is very simple; just execute the following command:

```
cd bsp/rockchip/rk2118
scons -j32
```

If the compilation is successful, you will obtain the following files:

```
-rw-r--r-- 1 rk rk  197632 Apr 11 07:08 rtt0.bin            # bin firmware for
cpu0
-rw-r--r-- 1 rk rk 1530528 Apr 11 07:08 rtt0.elf            # elf firmware for
cpu0, with symbol table
-rw-r--r-- 1 rk rk  110592 Apr 11 07:08 rtt1.bin            # bin firmware for
cpu1
-rw-r--r-- 1 rk rk  784044 Apr 11 07:08 rtt1.elf            # elf firmware for
cpu1, with symbol table
```

3.1.3.3 Generate root.img

We package some resource files (such as prompt tone files) into a FAT filesystem image, `root.img`.

This step is optional, and the `build.sh` script automatically generate the root image based on whether the root partition information is included in the firmware configuration file `setting.ini` or `package-file`.

The command to generate `root.img` is:

```
# For automotive (SPI Nor)
./mkroot.sh resource/userdata/normal/ board/adsp_demo/setting.ini

# For consumer products (SPI Nor/Nand/EMMC)
./mkroot.sh resource/userdata/normal/ board/evb/package-file

# The directory resource/userdata/normal/ contains the resource files that will
be packaged into root.img
```

3.1.3.4 Packing

The packaging process is to integrate the previously compiled firmware, including CPU (NPU firmware is temporarily placed with CPU) and DSP firmware, as well as the loader, TFM, etc., into one image file, either `Firmware.img` or `update.img`. The final flashing process uses this image.

The packaging command is as follows:

```
# For Automotive (SPI Nor)
./mkimage.sh board/adsp_demo/setting.ini

# For Consumer Products (SPI Nor/Nand EMMC)
./mkimage.sh board/evb/package-file

# Where board/adsp_demo/setting.ini provides the partition configuration
information for the Firmware.img firmware.
# Where board/evb/package-file provides the packaging configuration information
for the Update.img firmware.
```

3.2 DSP Firmware Compilation

Refer to <Rockchip_HiFi4_Quick_Start_EN.pdf>

The SDK comes with a precompiled DSP firmware located at `components/hifi4/rtt/dsp_fw/evb_soundbar_demo/`.

The corresponding DSP project and source code can be found at `components/hifi4/samples/rk2118/evb_partybox_demo/`. For specific functionality, you can refer to the `readme.txt` file located in this directory.

To compile firmware with the `evb_partybox_demo` functionality, you can use the following commands:

```
./build.sh board/evb/package-file
./build.sh board/evb1_v20/package-file
```

3.3 Firmware Format

3.3.1 Firmware.img

Used in automotive products with SPI Nor, it has the following characteristics:

1. Partition filled with redundant data, `Firmware.img` consists of valid firmware + redundant firmware. A complete `Firmware.img` is upgraded without support for upgrading only valid firmware.
2. Supports **RK partition** table.

The partition configuration information for `Firmware.img` is defined in `setting.ini`, as shown below:

```
[UserPart4]
Name=cpu0_os_a      # Partition name
Type=0x200
PartOffset=0x300    # Position of the partition in the flash,
                    # in sectors (512 bytes per sector)
PartSize=0xc00      # Partition size, in sectors (512 bytes per sector)
Flag=               # Flags, refer to the header of setting.ini for detailed
information
File=../../Image/rtt0.img    # Firmware file stored in the partition
```

3.3.2 update.img

Used in consumer products with SPI Nor/Nand/eMMC, it has the following characteristics:

1. `update.img` is valid firmware, and final upgrades are done by partition.
2. Supports **GPT partition**.

The partition configuration information for `update.img` is defined in `parameter.txt`, as shown below:

```
CMDLINE:
mtdparts=rk29xxnand:0x00000100@0x00000080(loader),0x00000100@0x00000180(cpu0_sec
ure_a),0x00000080@0x00000280(cpu1_secure),0x00000400@0x00000300(cpu0_os_a),0x000
00800@0x00000700(dsp0),0x00000800@0x00000f00(dsp1),0x00000800@0x00001700(dsp2),0
x00000100@0x00001f00(rkstudio),0x00001000@0x00002000(root),-
@0x00003000(userdata:grow)

# Example: 0x00000400@0x00000300(cpu0_os_a)
# The format is PartSize@PartOffset(Name), where PartSize and PartOffset are in
sectors (512 bytes per sector).
# Each partition is separated by a comma, and the PartOffset of the next
partition is calculated as the previous PartOffset + PartSize.
# The last partition doesn't need a specified PartSize as the software will
calculate it based on the flash size.
```

The packaging configuration file is called `package-file`, and it specifies the firmware components to be included in `update.img`.

3.4 File System Description

The SDK includes the following file systems, currently supporting only the FAT file system:

- **root**

Mounted at the root directory `/`, used to store resource files (such as prompt sounds). It is a read-only file system and will be updated during firmware upgrades. The mount information is configured in `board/common/mnt_base.c` under `mount_table`, and the DFS module automatically mounts it at boot.

- **userdata**

Mounted at the `/data` directory, used to store app configuration files. It is a read-write file system and will not be updated during firmware upgrades. The mounting code is located in `bsp/rockchip/rk2118/applications/userdata_mnt.c`.

- **udisk**

A USB disk, by default mounted at the `/udisk` directory. The mount point can be configured using `CONFIG_UDISK_MOUNTPOINT`.

- **sdcard**

An SD card, mounted at `/sdcard`. The mount information is configured in `board/common/mnt_base.c` under `mount_table`, and the DFS module automatically mounts it at boot.

You can check the file system mounting status using the `mount` command.

```
filesystem  device  mountpoint
----------  ------  ----------
elm         root    /
elm         userda  /data
elm         ud0-0   /udisk
```

4. Firmware Flash

4.1 Note

- Since the printed uart and the programmed uart are the same, you need to ensure that the serial port printing client is disconnected first when programming.
- USB cannot be plugged in when programming uart

4.2 Windows Upgrade Tool

4.2.1 Firmware.img

The tool is located at bsp/rockchip/tools/SocToolKit_v1.9_20240130_01_win.zip

Before programming, make sure the device has been switched to maskrom mode (Press and hold the MASKROM button on the development board, then press the RESET button to power on). Currently, Windows supports uart and usb programming methods. The first step is to select the programming method. For the uart method, you need to specify the serial port number and baud rate. Currently, the highest supported baud rate programming is 1.5M. It also supports full firmware programming and partition programming. The full firmware programming method is as follows:

step1: Choose the correct serial port

step2: Set the baud rate to 1500000

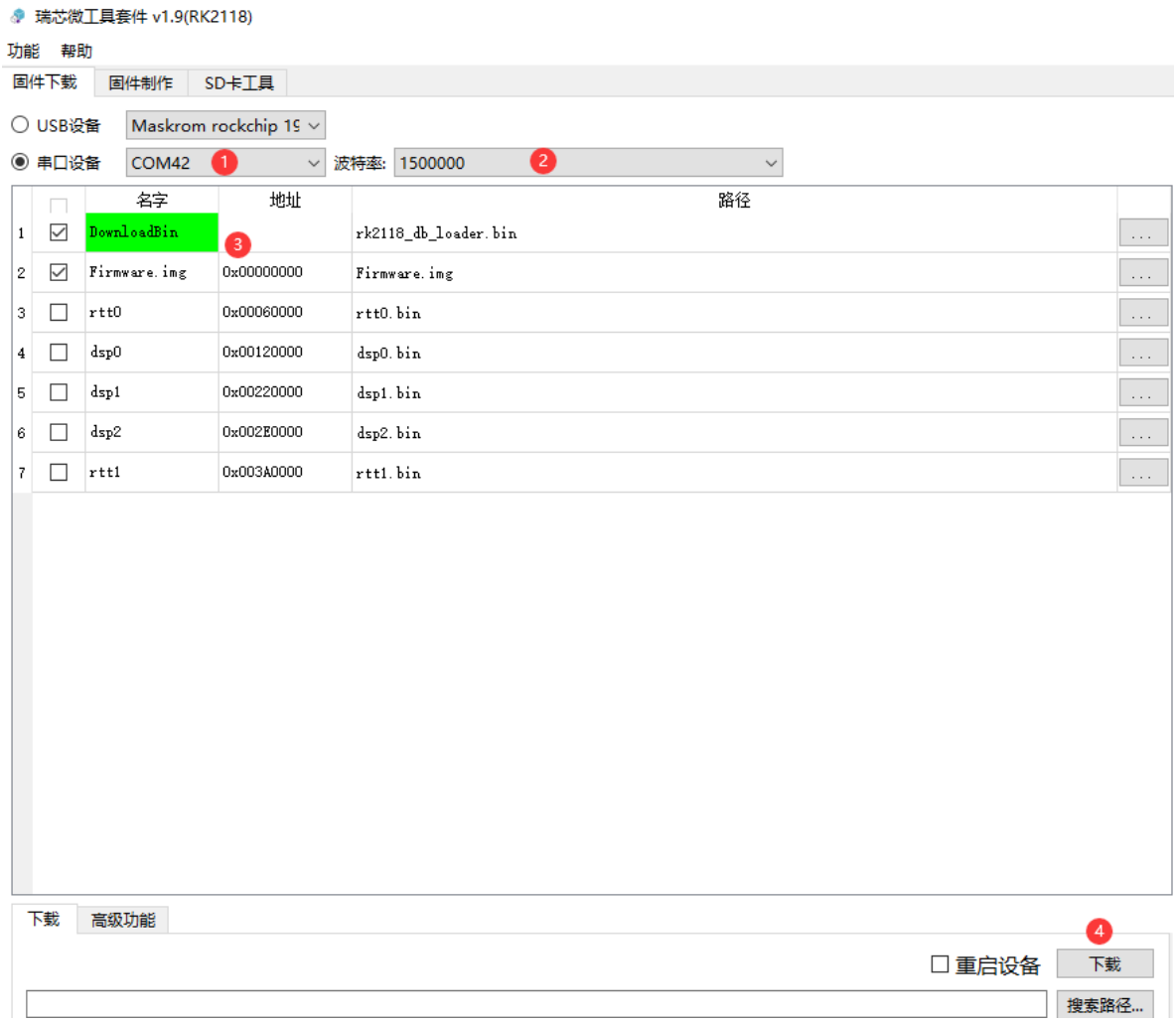
step3: To add firmware: Right-click in a blank area, select 'Add', and then fill in the name, address, and path. After that:

Select loader: bsp/rockchip/rk2118/rkbin/rk2118_db_loader.bin

Select Firmware.img: bsp/rockchip/rk2118/Image/Firmware.img, the address is 0x0

Right-click and choose 'Save Configuration' to your own config_xxx.json, so that you can directly use the saved firmware configuration next time.

step4: Start downloading

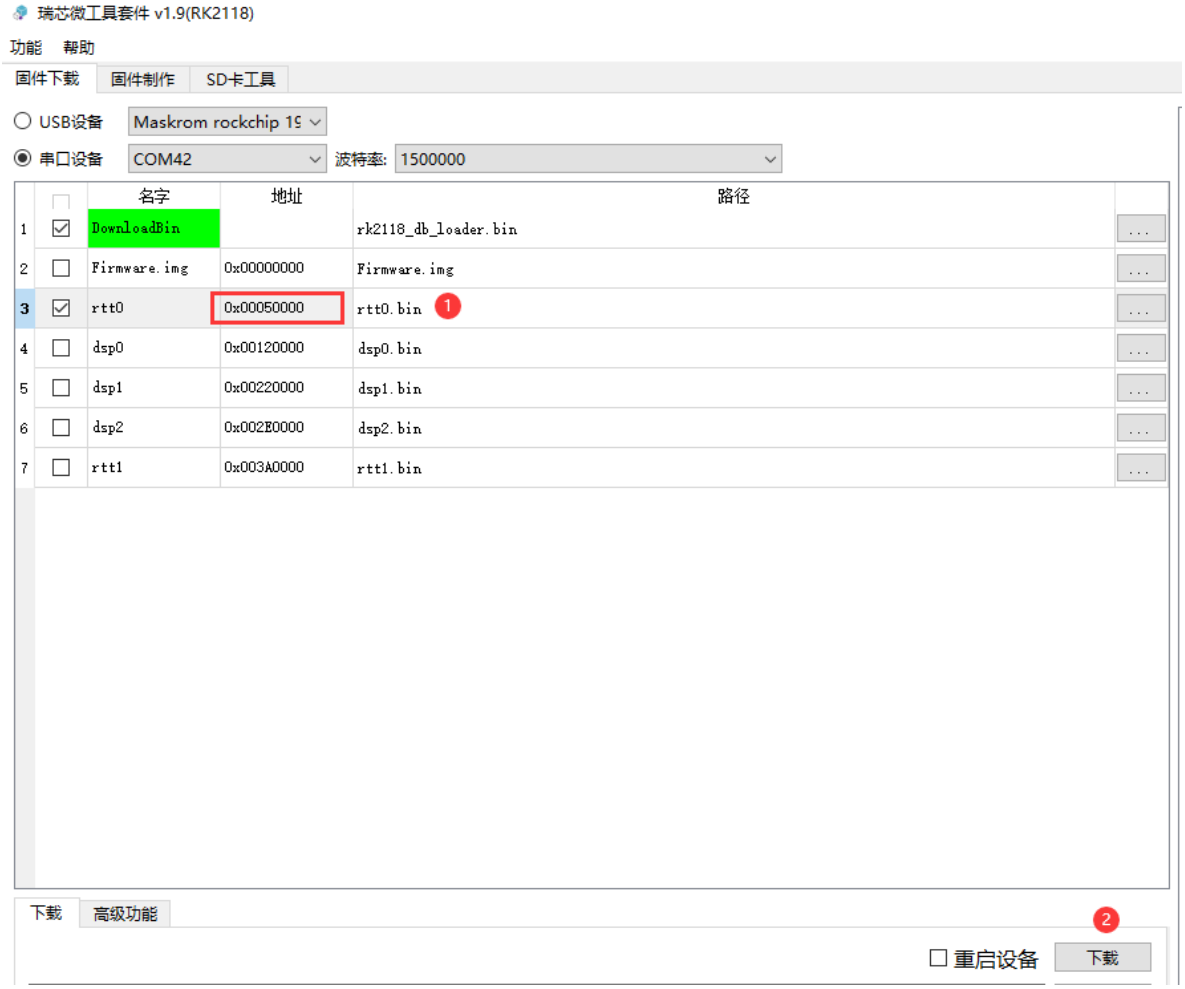


If you want to burn a specified partition (For example, if only rtt0.bin is modified, there is no need to re-upgrade Firmware.img, which can enhance efficiency), you need to specify the address of the partition. The burning method is as follows:

Select loader: bsp/rockchip/rk2118/rkbin/rk2118_db_loader.bin

Select the partition to be upgraded, such as rtthread: bsp/rockchip/rk2118/rtt0.bin, and find the address from the corresponding setting.ini, for example:

```
[UserPart3]
Name=OSA
Type=0x200
PartOffset=0x280  // the location of rtt0.bin
                  // the unit is sector, which needs to be multiplied by 512 to
convert to bytes, such as 0x280 X 512 = 0x5000
PartSize=0xa00
Flag=
File=../../rtt0.bin
```

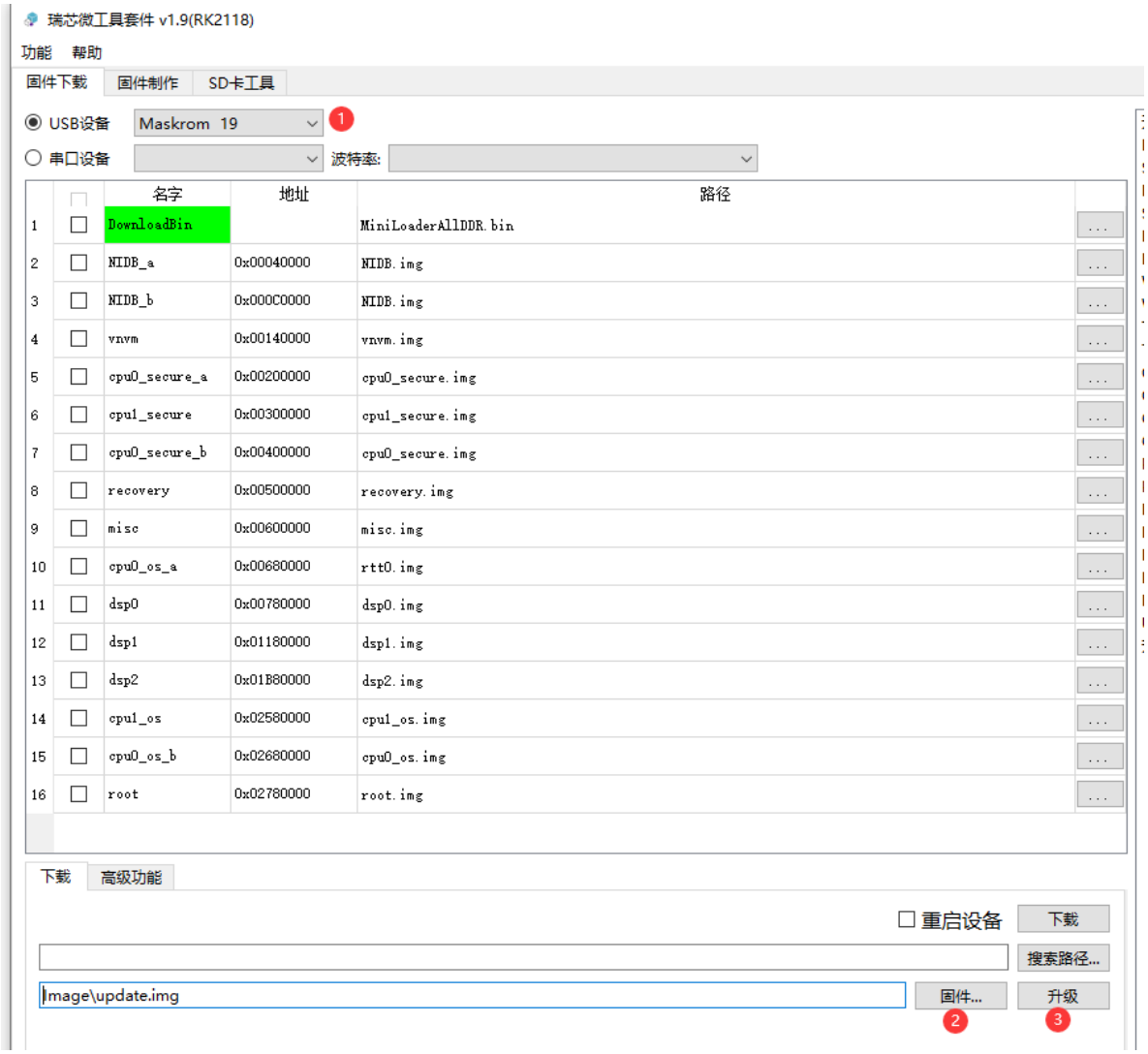


4.2.2 update.img

Step 1: Enter USB upgrade mode

Step 2: Select update.img

Step 3: Start the upgrade

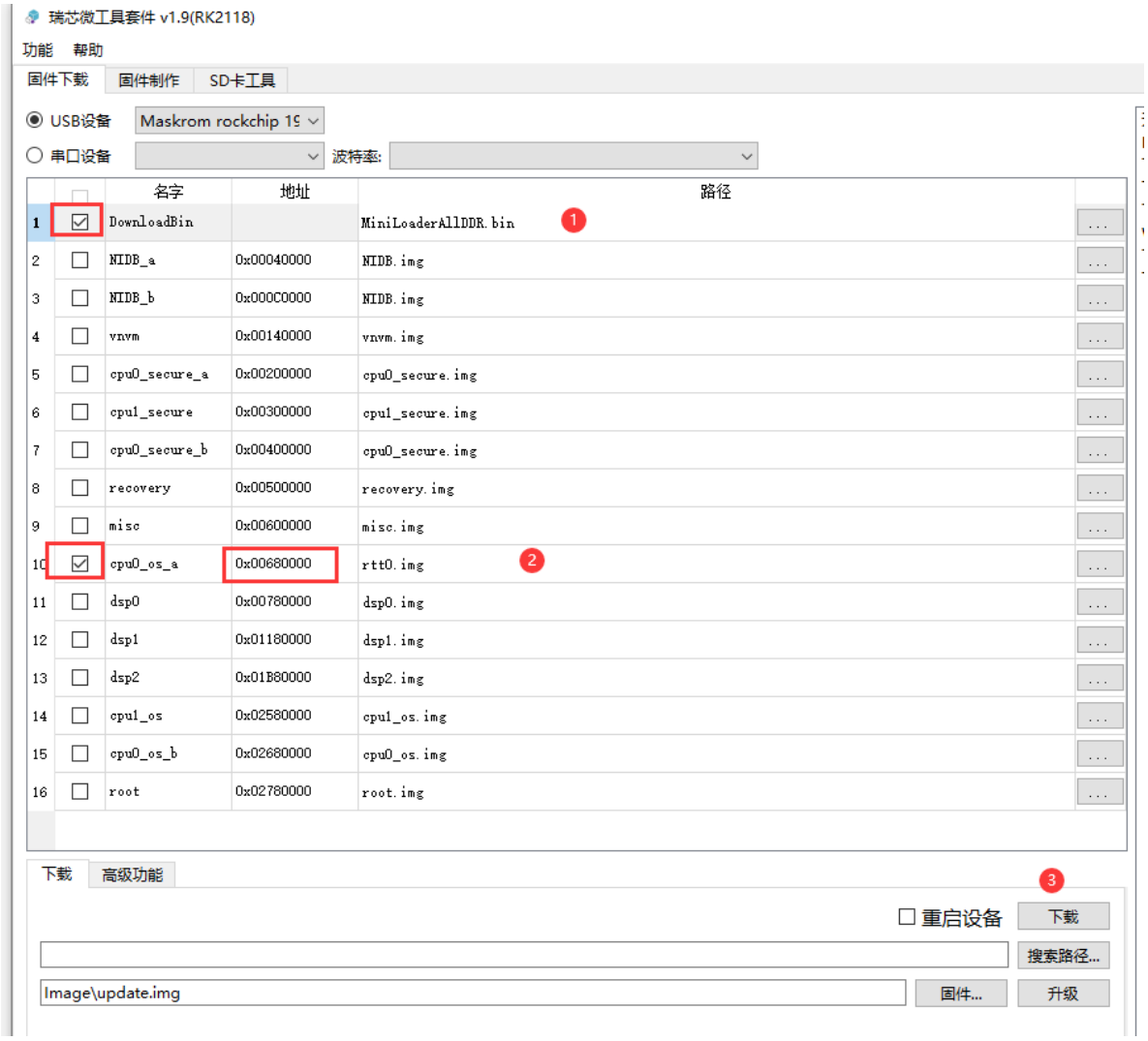


You can also upgrade a single image, such as rtt0.img.

Step 1: Select the loader.

Step 2: Select rtt0.img and ensure the correct address is entered, as per the previous section.

Step 3: Choose “Download” to start the upgrade.



4.3 Linux Upgrade Tool and Command

4.3.1 UART Programming

The default ttyUSB device under Linux does not have write permissions. Modify the permissions through sudo chmod 666 /dev/ttyUSBx (x is your serial port number) before burning.

If you don't want to change the serial port permissions every time, you can add default write permissions in the following way:

```
sudo vi /etc/udev/rules.d/70-rk.rules
# Add the following content when creating, save and exit
KERNEL=="ttyUSB[0-9]*", MODE="0666"
```

The programming method is as follows:

```
sudo chmod 666 /dev/ttyUSB0
# First switch to maskrom and burn db loader
../tools/upgrade_tool/upgrade_tool db /dev/ttyUSB0 ./Image/rk2118_db_loader.bin
# Complete firmware programming
../tools/upgrade_tool/upgrade_tool wl /dev/ttyUSB0 0 ./Image/Firmware.img
# After the complete firmware is burned, the development process can only burn
the updated firmware. For example, the following command is to burn the rtt
firmware of cpu0 alone. The third parameter is the firmware location. By viewing
the UserPart3 partition in setting.ini, the PartOffset=0x280, so fill in 0x280
here
../tools/upgrade_tool/upgrade_tool wl /dev/ttyUSB0 0x280 ./rtthread.bin
```

4.3.2 USB Programming

The programming method is as follows:

```
# First switch to maskrom and burn db loader
../tools/upgrade_tool/upgrade_tool db ./Image/rk2118_db_loader.bin
# Complete firmware programming
../tools/upgrade_tool/upgrade_tool wl 0 ./Image/Firmware.img
# After the complete firmware is burned, the development process can only burn
the updated firmware. For example, the following command is to burn the rtt
firmware of cpu0 alone. The location information needs to be kept equal to the
PartOffset of the partition in setting.ini.
../tools/upgrade_tool/upgrade_tool wl 0x280 ./rtthread.bin
```

5. Run and Debug

5.1 System Start

There are several ways to start the system:

1. After firmware upgrade, restart automatically;
2. Plug in power supply and start directly;
3. For devices with battery power supply, press Reset button to start;

5.2 System Debug

RK2118 supports serial port debugging. Different hardware devices have different serial port configurations.

The serial communication configuration information is as follows:

Baud rate: 1500000

Data bits: 8

Stop bit: 1

Parity: none

Flow control: none

The following picture indicates entering debugging successfully:

```
Booting TF-M 20240303
                     clk_init: PLL_GPLL = 800000000
clk_init: PLL_VPLL0 = 1179648000
clk_init: PLL_VPLL1 = 903168000
clk_init: PLL_GPLL_DIV = 800000000
clk_init: PLL_VPLL0_DIV = 294912000
clk_init: PLL_VPLL1_DIV = 150528000
clk_init: CLK_DSP0_SRC = 400000000
clk_init: CLK_DSP0 = 700000000
clk_init: CLK_DSP1 = 200000000
clk_init: CLK_DSP2 = 200000000
clk_init: ACLK_NPU = 400000000
clk_init: HCLK_NPU = 133333333
clk_init: CLK_STARSE0 = 400000000
clk_init: CLK_STARSE1 = 400000000
clk_init: ACLK_BUS = 266666666
clk_init: HCLK_BUS = 133333333
clk_init: PCLK_BUS = 133333333
clk_init: ACLK_HSPERI = 133333333
clk_init: ACLK_PERIB = 133333333
clk_init: HCLK_PERIB = 133333333
clk_init: CLK_INT_VOICE0 = 49152000
clk_init: CLK_INT_VOICE1 = 45158400
clk_init: CLK_INT_VOICE2 = 98304000
clk_init: CLK_FRAC_UART0 = 64000000
clk_init: CLK_FRAC_UART1 = 48000000
clk_init: CLK_FRAC_VOICE0 = 24576000
clk_init: CLK_FRAC_VOICE1 = 22579200
clk_init: CLK_FRAC_COMMON0 = 12288000
clk_init: CLK_FRAC_COMMON1 = 11289600
clk_init: CLK_FRAC_COMMON2 = 8192000
clk_init: PCLK_PMU = 100000000
clk_init: CLK_32K_FRAC = 32768
clk_init: CLK_MAC_OUT = 50000000
clk_init: MCLK_PDM = 100000000
clk_init: CLKOUT_PDM = 3072000
clk_init: MCLK_SPDIFTX = 6144000
clk_init: MCLK_OUT_SAI0 = 12288000
clk_init: MCLK_OUT_SAI1 = 12288000
clk_init: MCLK_OUT_SAI2 = 12288000
clk_init: MCLK_OUT_SAI3 = 12288000
clk_init: MCLK_OUT_SAI4 = 12288000
clk_init: MCLK_OUT_SAI5 = 12288000
clk_init: MCLK_OUT_SAI6 = 12288000
clk_init: MCLK_OUT_SAI7 = 12288000
clk_init: SCLK_SAI0 = 12288000
clk_init: SCLK_SAI1 = 12288000
clk_init: SCLK_SAI2 = 12288000
clk_init: SCLK_SAI3 = 12288000
clk_init: SCLK_SAI4 = 12288000
clk_init: SCLK_SAI5 = 12288000
clk_init: SCLK_SAI6 = 12288000
clk_init: SCLK_SAI7 = 122880?

 \ | /
- RT -     Thread Operating System
 / | \     4.1.1 build Mar 15 2024 17:10:53
 2006 - 2022 Copyright by RT-Thread team
[I/I2C] I2C bus [i2c2] registered
[DSP0 INFO] Hello RK2118 dsp0, build Mar 15 2024 16:58:49
[DSP1 INFO] Hello RK2118 dsp1, build Mar 15 2024 16:59:09
[DSP2 INFO] Hello RK2118 dsp2, build Mar 15 2024 16:59:30
delay 1s test start
msh >delay 1s test end
```

6. Development Guide

Please refer <Rockchip_Developer_Guide_RK2118_EN.pdf>